HACKATHON · SOLO

AI *Reassignment Engine*

5 hours
Solo · Individual
Spring Boot 3.x
React 18 or Angular 17
117 pts total

THE SCENARIO

ZipRun runs a fleet of delivery agents across the city. Orders are manually assigned to agents at the start of each shift — that works fine until something goes wrong mid-day. An agent calls in sick. A bike breaks down. Someone becomes unavailable after already being handed a set of orders.

When that happens today, an ops manager notices it, opens a spreadsheet, figures out who has capacity, and reassigns manually. It works — but it's slow, it depends on one person paying attention, and it fails silently when no one is watching.

That's the problem you're solving. When an agent goes offline, your system picks it up automatically, figures out which orders are affected, uses AI to recommend the best reassignment, and puts that recommendation in front of ops for a quick approval — without anyone having to ask for it.

> SCOPE FOR THIS SPRINT
>
> You're building a reactive reassignment engine — not a full dispatch system. Order creation workflows, zone management, and SLA dashboards are coming in sprint 2 and 3. Your job right now is to nail the reassignment loop and build the foundation that makes those extensions straightforward to add. The Extensions tab has the full roadmap.

WHAT YOU'RE BUILDING

- **A domain model** — clean entities with the right state machines to support reassignment today and sprint 2/3 extensions without structural rework
- **A routing engine** — a pluggable strategy interface with one rule-based implementation and one AI-powered one, switchable at runtime without a restart
- **An AI advisor** — an LLM integration that looks at an order, the available agent roster, and the situation context, then comes back with a recommended agent, a confidence score, and a plain-English explanation ops can actually act on
- **An agentic re-planning loop** — when an agent goes offline, the system spots the affected orders, re-runs routing, and queues reassignment suggestions for ops to approve — all without anyone clicking a button
- **A minimal ops interface** — built in React or Angular, showing pending reassignment suggestions with AI reasoning and accept/reject controls

A FEW THINGS WORTH KNOWING

- This is a solo effort — every line of code and every architectural call is yours alone, and that's a good thing. You get to make real decisions.
- AI coding tools — Cursor, Copilot, Claude — are fair game and genuinely encouraged. We use them too. The only thing we'd ask is that you understand what you've shipped, because we'll walk through it together afterwards.
- The walkthrough after submission is a conversation, not an exam. We'll trace a couple of paths through the code together, so write things you can explain, not just things that work.
- What you're submitting: a public GitHub repo, a 5-minute demo video, and an `ADR.md`. The ADR is the piece most people underestimate — give it the same attention as the code.
- Two addenda are at the end of this brief — a seed script and an LLM gateway utility — to save you setup time on things that don't carry evaluation signal. Use them, adapt them, or ignore them entirely.

T-1

Domain model & API

`Backend` `20 pts` `~50 min`

Start here — get the foundation right and the rest of the build flows naturally. Think about the state machines before you model the entities; the transitions matter more than the field list.

Order — captures identity, which agent it's assigned to, and where it is in its lifecycle: `ASSIGNED → REASSIGNMENT_PENDING → REASSIGNED → DELIVERED`

Agent — captures identity, current order load, and availability: `AVAILABLE`, `BUSY`, `OFFLINE`

ReassignmentSuggestion — this one threads through everything, so design it carefully. It needs: which order it relates to, which agent is recommended, the AI's confidence score (0.0–1.0), the AI's plain-English reasoning, its own status (`PENDING → ACCEPTED | REJECTED`), and what triggered it (`INITIAL` or `AGENT_OFFLINE`). That `triggerReason` field connects your domain model to the agentic loop in T-4 and to the re-plan badge in the UI — worth getting in early.

Four endpoints to wire up:

- `POST /orders` — create an order pre-assigned to an agent, simulating the morning manual assignment
- `GET /orders?status=` — list orders, filterable by status
- `PATCH /agents/{id}/status` — update agent availability; this is what fires the agentic loop
- `PATCH /suggestions/{id}` — accept or reject a suggestion, with the new status in the request body

> DESIGNING FOR WHAT'S COMING
>
> Sprint 2 brings zone awareness, capacity constraints, weight classes, and SLA deadlines. You're not building those today, but a couple of minutes thinking about where they'd live in your model is well spent. A nullable placeholder field now costs nothing. A migration in sprint 2 costs a sprint. Your ADR should touch on at least one of these.

> SEED DATA

> Addendum A has a ready-to-run seed script — 5 agents and 8 pre-assigned orders. Use it as-is or adapt it to your schema. No need to write seed data from scratch.

T-2

Pluggable routing engine

Backend · Design Pattern | 25 pts | ~55 min

The routing engine picks the best available agent for a given order. The interesting design challenge isn't the picking logic itself — it's making the engine genuinely replaceable without touching existing code when sprint 2 introduces a new strategy.

Define a routing contract that takes an order and a list of available agents and returns an ordered list of recommendations. Then build two implementations against it:

A rule-based strategy — recommend the available agent carrying the fewest active orders. Simple, deterministic, no external dependencies, and a solid fallback when the AI is unavailable.

An AI strategy — that's T-3, but design the interface with it in mind from the start.

The active strategy should be switchable via config at runtime — no code change, no restart. How you wire that switchability is a genuine architectural decision worth an ADR entry.

Wire up `POST /orders/{id}/suggest` — runs the active strategy, persists a `ReassignmentSuggestion`, and returns it.

> TWO CALLERS, ONE INTERFACE
>
> Your routing contract gets called from two places — an HTTP endpoint for on-demand suggestions, and an async event handler in the agentic loop. Design it so both callers can use it without modification. That constraint is worth factoring in before you get to T-4, not after.

> SPRINT 2 HEADS UP
>
> Sprint 2 adds a `ZoneAffinityStrategy`. Ideally, adding it means implementing your contract and registering it — nothing more. Where does that extension point sit in your design?

T-3

AI routing strategy

AI Integration | 25 pts | ~55 min

This is the AI implementation of your routing contract. It calls an LLM, gets back a recommendation, validates it, and either uses it or falls back gracefully.

Give the LLM what it needs to reason well: the order details, a snapshot of all available agents with their current load, and the trigger context — is this a first assignment, or a recovery from an agent going offline? Those are different situations and the model should know which one it's in.

Expect back: a recommended agent identifier, a confidence score between 0 and 1, and a plain-English explanation an ops person can actually act on. Store that reasoning in the suggestion and display it verbatim in the UI — that's the part ops reads.

A few things worth getting right:

- Validate the returned agent identifier against your actual agent list — LLMs occasionally hallucinate identifiers
- Handle timeouts, quota errors, and unparseable responses — fall back to the rule-based strategy and log what happened
- Keep AI calls off the critical path — the main request flow shouldn't wait on an LLM

> TWO PROMPTS, NOT ONE
>
> An initial suggestion and a re-plan after an agent goes offline are genuinely different situations. The re-plan prompt should tell the model what happened — which agent failed, how many orders are now stranded, and

that this is a recovery. If your two prompts are essentially the same document with one field added, the model isn't getting the context it needs to reason differently. Both prompts will come up in the walkthrough.

ADDENDUM B

Addendum B has a working `callLLM(String prompt)` utility covering the HTTP wire format for Gemini, Groq, and Ollama. Prompt construction, response parsing, validation, and fallback are all yours — the addendum just removes the provider-specific boilerplate.

BONUS · +5 PTS

If time allows: `POST /orders/{id}/suggest/stream` — stream the LLM's reasoning token by token via Server-Sent Events. Ops sees the AI thinking in real time before the suggestion lands.

T-4

Agentic re-planning loop

Agentic | 15 pts | ~45 min

This is the heart of the system — the part that makes it agentic rather than just a fancy form. When an agent goes offline, the system should notice, figure out which orders are affected, and queue reassignment suggestions automatically. Nobody clicks a button. It just happens.

Concretely: when `PATCH /agents/{id}/status` sets an agent to `OFFLINE`, a background process identifies all orders currently assigned to that agent, runs the active routing strategy on each one, and creates a `ReassignmentSuggestion` with `triggerReason = AGENT_OFFLINE` for each.

A few things to get right:

- The PATCH endpoint returns immediately — re-planning is fully async and never blocks the response
- Suggestions surface in the UI on the next poll — ops doesn't need to do anything to see them

- If a `PENDING` suggestion with `triggerReason = AGENT_OFFLINE` already exists for an order, skip creating another one — the same agent flipping offline twice shouldn't produce duplicates
- If the AI strategy fails during async re-planning, fall back to rule-based and surface the suggestion anyway — a silent drop is worse than a rule-based recommendation

> THE LOOP IN PLAIN TERMS
>
> Observe (agent goes offline) → reason (which orders are now stranded?) → act (queue suggestions) → checkpoint (ops approves). That last step is important — the system queues suggestions, it doesn't auto-assign. Ops makes the final call, and that's intentional. See the Agentic Layer tab for the deeper design questions this raises.

> WIRING SUGGESTION
>
> Spring's `@EventListener` with `@Async`, or a domain event via `ApplicationEventPublisher`, are clean ways to decouple the status update from the re-planning. A scheduled poller isn't quite right here — the loop should fire because something changed, not because a timer ticked. Whichever approach you take, it's worth an ADR entry.

T-5

Ops interface

`React 18 or Angular 17` `12 pts floor · +8 pts ceiling` `~40 min`

Build the interface an ops person would open when they get a reassignment alert. It doesn't need to be beautiful — it needs to be usable and it needs to show the agentic loop working end-to-end. Build it in React 18 or Angular 17 — your choice, documented in the ADR.

The floor — what everyone should ship:

- A list of orders in `REASSIGNMENT_PENDING` state, with the current suggestion shown inline — recommended agent, confidence score, and the AI's reasoning verbatim
- Accept and Reject buttons wired to `PATCH /suggestions/{id}`

- A visual badge distinguishing agentic re-plans (`triggerReason = AGENT_OFFLINE`) from manually requested suggestions — this is what makes the loop visible
- An agent list showing current status — AVAILABLE, BUSY, OFFLINE — clearly differentiated
- Polling or a manual refresh button so suggestions appear without a page reload
- Loading states and sensible error handling — these aren't extras, they're just good engineering

The ceiling — if you have time and want to go further:

- Full dispatch board showing all orders across all statuses
- SLA countdown per order with green / amber / red colour coding
- Agent load visualisation — orders carried vs capacity
- Zone-aware roster showing current zone per agent

> ON PRIORITIES
>
> This is primarily a backend and systems design screen. The UI's job is to show the system working — especially the re-plan badge appearing after an agent goes offline. If you're short on time, a clean functional floor beats an ambitious ceiling that cost you something in the backend. Document your call in the ADR.

T-6

ADR + live walkthrough

Architecture ownership | 20 pts

The ADR is your chance to show the thinking behind the code — not what you built, but why, what you considered, and what you consciously left for later.

Aim for four solid entries, each following: **Context → Options considered → Decision → Tradeoffs accepted**. The four areas we'd love you to cover:

- Where does routing logic live, and why there?
- How does strategy switchability work, and what were the alternatives?

- How does the system handle an unavailable LLM?
- How is the agentic loop triggered and kept off the request path?

Two entries that tend to carry a lot of weight:

- **Extensibility** — pick one item from the sprint 2/3 roadmap and point to the extension seam in your actual code. Saying "it's extensible" is nice; showing where is much better.
- **Deliberate exclusions** — what did you choose not to build, and why? A priority rationale lands differently than a time excuse. "I deferred the full dispatch board because the agentic loop is a correctness requirement and the board is a visibility enhancement" is a decision.

The walkthrough is a conversation — we'll trace an order through the system together, and then follow what happens when the assigned agent goes offline. Come ready to talk through the code, not recite it.

> ONE PIECE OF PRACTICAL ADVICE
>
> Write each ADR entry right after making the corresponding decision, not at the end of the session when you're tired and the clock is running out. The reasoning is freshest in the moment, and it shows.

The ADR — Architecture Decision Record — is one of the most important things you'll submit. It's not documentation written after the fact. It's a record of the thinking that shaped your code — captured in the moment, entry by entry, as you make each call. This tab explains the format, shows you a worked example, and lists the five decisions you need to cover.

WHAT AN ADR IS

An ADR is a short document that records a single architectural decision. Not a summary of everything you built — just one decision per entry. The goal is to capture enough context that someone reading it six months later (or a reviewer reading it in the walkthrough) understands not just *what* you chose but *why*, and what you accepted by making that choice.

The format is deliberately simple: four fields, each a short paragraph. You don't need headers, diagrams, or lengthy prose. A good ADR entry takes five to ten minutes to write well — which is why the best approach is to write each one right after making the corresponding decision, not at the end when the session is winding down.

THE FORMAT

CONTEXT	What is the situation that requires a decision? What problem are you solving? What constraints are you working within? Keep this to two or three sentences — enough for someone unfamiliar with the codebase to understand why this decision needed to be made at all.
OPTIONS	What approaches did you consider? List at least two — even if one was clearly better, naming the alternatives shows you thought it through rather than defaulting to the first thing that came to mind. A single sentence per option is enough.
DECISION	What did you choose and why? This is the heart of the entry. The reason should connect to something meaningful — testability, extensibility, simplicity, the sprint 2 roadmap — not just personal preference.
TRADEOFFS	What did you give up or accept by making this choice? Every decision has a cost — naming it honestly is a sign of engineering maturity. If you can't name a tradeoff, you probably haven't thought the decision through fully.

A WORKED EXAMPLE

EXAMPLE — ADR-2: HOW IS THE ROUTING STRATEGY MADE SWITCHABLE AT RUNTIME?

> **CONTEXT**

The routing engine supports multiple strategies — a rule-based strategy and an AI strategy — and the active strategy needs to be switchable at runtime without a restart. The same routing contract is called from two places: an HTTP endpoint for on-demand suggestions and an async event handler in the agentic re-planning loop. Sprint 2 will introduce a third strategy (`ZoneAffinityStrategy`), so the wiring mechanism needs to accommodate new strategies without changing existing code.

OPTIONS CONSIDERED

(a) Spring `@Qualifier` with a config property — straightforward but requires a restart to switch and doesn't scale cleanly to dynamic registration.
(b) Auto-wired `Map<String, RoutingStrategy>` bean map — Spring populates the map automatically; the active strategy is selected at call time by reading a config property. Adding a new strategy means implementing the interface and registering the bean — no existing code changes.
(c) Manual factory with a switch statement — explicit but requires modifying the factory every time a new strategy is added.

DECISION

Chose option (b) — auto-wired bean map keyed by strategy name. The active strategy name is read from `routing.strategy` in `application.properties`, which can be changed via environment variable without a restart. The map is injected into both the HTTP endpoint handler and the agentic event listener, so both callers get the same active strategy without additional wiring. Adding `ZoneAffinityStrategy` in sprint 2 requires implementing the interface and annotating the bean — no changes to selection logic.

TRADEOFFS ACCEPTED

The bean map approach is slightly less explicit than a factory — a reader needs to know that Spring auto-populates the map by bean name. This is a minor readability cost. We also lose compile-time guarantees that the configured strategy name actually exists; if `routing.strategy` is misconfigured, the failure is at runtime. Added a startup validation check to surface this immediately rather than on the first request.

THE FIVE DECISIONS TO COVER

Your `ADR.md` needs at least four solid entries. The five decisions below are the ones we'd expect to see — the first four are non-negotiable, the fifth is the one that tends to carry the most weight in the walkthrough.

1 **Where does routing logic live?**

Name the pattern — Service Layer, Domain Model, Application Service. Explain what boundary it enforces and why you didn't put the logic somewhere else. A service class that quietly grows to hold routing logic, event publishing, and persistence all at once is a design smell worth naming and avoiding.

2 **How does runtime strategy switching work?**

The worked example above covers this one. Your version should reflect your actual implementation — not the example. If you chose a different approach, document why. The key things to address: how it works from both call paths (HTTP and async), and how sprint 2's third strategy plugs in.

3 **How does the system stay healthy when the LLM is unavailable?**

LLM calls fail in several distinct ways — timeouts, malformed JSON, hallucinated agent IDs, quota exhaustion. Define how you handle each case and what the system does in response. The async re-plan path needs special attention: a failed AI call there should still produce a rule-based suggestion, not a silent drop.

4 **How is the agentic loop triggered and kept off the request path?**

The `PATCH /agents/{id}/status` endpoint must return quickly. Document the mechanism you chose for decoupling the re-planning — `@EventListener`, `ApplicationEventPublisher`, thread pool, or something else. Explain why you chose it over the alternatives, and address what happens when the async re-plan itself fails.

5 **What did you design to extend, and what did you deliberately leave for later?**

Two parts. First — pick one sprint 2 or sprint 3 feature and point to the extension seam in your actual code. The interface, field, or abstraction that makes it addable without restructuring what's already there. Second — name at least one thing

you chose not to build, and frame it as a priority call. "I deferred X because Y is a correctness requirement and X is an enhancement" is a decision. "Ran out of time" isn't.

> **A PRACTICAL TIP**
>
> Keep your **ADR.md** open in a split pane throughout the session. Write the entry for each decision as you make it — five minutes now while the reasoning is fresh is worth far more than twenty minutes at the end trying to reconstruct it. The walkthrough will test whether the ADR reflects your actual thinking or was assembled retrospectively. It tends to be obvious which one it is.

Here's what's on the roadmap for sprints 2 and 3. You're not building any of this today — but it's worth having in your head when you're making domain model and interface decisions. The goal is that when these features arrive, adding them feels natural rather than surgical.

SPRINT 2 — ZONE & CAPACITY

- Agents carry a `currentZone` — routing starts preferring agents already near the pickup zone
- `ZoneAffinityStrategy` becomes a third routing option alongside rule-based and AI
- Agents get a `maxCapacity` — routing respects it; no agent goes over their limit
- Orders carry a `weightClass` (`LIGHT` / `HEAVY`) — heavy orders go to capable agents only
- Orders get a `pickupZone` and `dropoffZone` — used by zone-aware strategies

SPRINT 3 — PROACTIVE & FULL DISPATCH

- SLA deadlines on orders — the agentic loop fires proactively when an order approaches breach, not just on agent OFFLINE events
- Priority tiers — premium orders jump the queue regardless of active strategy
- Full dispatch board — all orders across all statuses, real-time agent load, zone map
- Traffic-aware routing — AI strategy calls an external data source as a tool before recommending
- Multi-step re-plan — AI considers downstream queue pressure across all agents, not just the immediate order

> **HOW TO THINK ABOUT THIS**

> Three places where today's decisions can either welcome or resist these extensions: the domain model (does adding `currentZone` need a migration or just an activated placeholder?), the routing interface (does a third strategy mean implementing a contract, or modifying existing ones?), and the agentic loop (is the trigger mechanism general enough to handle an SLA event as naturally as an OFFLINE event?). Your ADR extensibility entry should address at least one of these.

AGENTIC LOOP — QUESTIONS WORTH THINKING THROUGH

T-4 is worth thinking through carefully before you build it. These are the questions your implementation should have good answers to — and ones we'll explore together in the walkthrough.

AGT-1 What actually makes this agentic rather than just automated?

A scheduled job that checks every few minutes for offline agents is automated. What makes a system agentic is the loop: observe a specific state change, reason about what it means, act within defined boundaries, then wait for a human to make the irreversible call. In this system — the trigger is the OFFLINE event (not a timer), the reasoning is about which specific orders are stranded (not all orders), the action is queuing suggestions (not assigning), and the checkpoint is ops approval. Keep all four parts of that loop in mind as you build T-4.

AGT-2 Where's the human checkpoint, and when might you remove it?

The checkpoint is the suggestion queue — the system proposes, ops disposes. Auto-assigning on re-plan isn't "more complete," it's a different system where the human is out of the loop. Your ADR should name the checkpoint and — this is the more interesting part — say under what conditions you'd be comfortable removing it. High confidence score? Imminent SLA breach? That reasoning shows you're thinking about the system's evolution, not just its current sprint.

AGT-3 Why do the two prompts need to be different?

A first assignment and a recovery from a failure are genuinely different situations. The model needs to know it's in recovery mode — which agent dropped out, how many orders are now stranded, and that previous assignments to that agent are void.

Handing the model the same context as a first assignment with a small note appended means it isn't getting what it needs to reason differently. Think of the re-plan prompt as a situation report, not an updated order form.

AGT-4 **What does idempotency mean here and why does it matter?**

If the same agent goes offline twice quickly, or two agents go offline at once affecting overlapping orders, a naive implementation fires duplicate suggestions. Before creating a new re-plan suggestion for an order, check whether a `PENDING` suggestion with `triggerReason = AGENT_OFFLINE` already exists for it — if so, skip it. This is a correctness concern, not a performance one, and it's worth showing in your repository query and mentioning in the ADR.

AGT-5 **Where does your design support the sprint 3 proactive loop?**

Sprint 3 introduces SLA-breach detection — the loop fires not just on OFFLINE events but when an order is getting close to its deadline. We'll ask about this in the walkthrough, but you're not expected to build it — just be able to point to where it would plug in. Is your event mechanism general enough to accept a new trigger type? Is your routing interface as comfortable being called from a scheduled monitor as from an event handler? That's the kind of forward thinking we enjoy exploring together.

ARCHITECTURE DECISIONS — ADR GUIDE

These are the architectural questions your ADR should address and that we'll dig into together in the walkthrough. There's no single right answer — a well-reasoned choice with honestly acknowledged tradeoffs is what we're looking for.

ADR-1 **Where does routing logic live?**

Controller, service, domain object, or a dedicated component? Name the pattern — Service Layer, Domain Model, Application Service — explain what it encapsulates, and say why you didn't put it somewhere else. A service that quietly accumulates routing logic, fallback logic, event publishing, and persistence is a design smell worth naming and avoiding.

ADR-2 **How does runtime strategy switching work?**

Spring `@Qualifier`, an auto-wired `Map<String, RoutingStrategy>` bean map, a manual factory, or a Strategy + Registry — each has different tradeoffs for testability, thread-safety, and extensibility. Worth keeping in mind that the strategy is called from both an HTTP endpoint and an async event handler, and that sprint 2 adds a third. How many files does adding it touch with your current approach?

ADR-3 How does the system stay resilient when the LLM is unavailable?

LLM calls fail in interesting ways — timeouts, quota exhaustion, malformed JSON, hallucinated agent identifiers. Worth defining how you handle each and what the system does in response. In the async re-plan context especially, a failed AI call should still produce a rule-based suggestion rather than silently dropping the re-plan. The reassignment flow should stay healthy regardless of what the LLM is doing.

ADR-4 How is the agentic loop triggered and kept off the request path?

The `PATCH /agents/{id}/status` endpoint needs to return quickly — re-planning happens somewhere else. Spring's `@EventListener` + `@Async`, a domain event via `ApplicationEventPublisher`, or a dedicated thread pool executor all work, each with different characteristics around error propagation and transaction boundaries. Whichever you pick, it's worth thinking through what happens when the async re-plan itself fails — does that failure get logged, retried, surfaced to ops?

ADR-5 What did you design to extend, and what did you save for later?

Two parts to this one. First — pick a sprint 2 or sprint 3 feature and point to the extension seam in your code. The interface, the field, the abstraction that makes it addable without restructuring what's already there. Second — name at least one thing you chose not to build. Frame it as a priority call: "I deferred X because Y is a correctness requirement and X is an enhancement" lands differently than "didn't have time."

SCORING

Here's how the session is scored. The rubric is here so you know where to invest your time and energy — not to add pressure. The areas that carry the most weight are the routing engine, AI integration, and the ADR and walkthrough, so keep those in focus even if other things get tight.

20	25	25	15	12	20	+8+5
DOMAIN MODEL & API	ROUTING ENGINE	AI INTEGRATION	AGENTIC LOOP	OPS INTERFACE FLOOR	ADR + WALKTHROUGH	UI CEILING · SSE BONUS

AREA	WHAT WE'RE LOOKING FOR	PTS
Entity design	Clean state machines, extension points visible for sprint 2 fields, model feels purposeful rather than accidental	8
API correctness	Right HTTP verbs and status codes, validation in the right place, errors come back in a structured shape	7
Persistence	JPA relationships correct, no obvious N+1 queries, the app runs from the README in under 5 minutes	5
Routing contract	Interface defined before implementations; works cleanly from both HTTP and async call paths; adding a third strategy touches minimal existing code	10
Runtime switchability	Strategy changes via config without a restart; mechanism makes sense from both call sites	8
Pattern justification	ADR entry names the pattern, defends it against alternatives, and connects it to the sprint 2 extensibility picture	7
Initial prompt quality	Gives the model what it needs — structured order context, full agent roster snapshot, typed output schema requested	8

AREA	WHAT WE'RE LOOKING FOR	PTS
Re-plan prompt quality	Genuinely different from the initial prompt; includes failure context — agent offline, orders affected, recovery framing	8
AI resilience	All failure modes handled explicitly; agent IDs validated before persisting; async re-plan failures produce a rule-based fallback rather than a silent drop	9
Agentic loop — trigger	Event-driven and non-blocking; fires on OFFLINE; idempotent on repeated triggers	7
Agentic loop — checkpoint	System queues suggestions and doesn't auto-assign; human checkpoint is clear in the code and named in the ADR	8
Ops interface floor	Reassignment list with inline AI reasoning; accept/reject controls wired up; re-plan badge visible; agent list with status; polling or refresh; loading and error states present	12
ADR quality	Four solid entries with context / options / decision / tradeoffs; extensibility entry points to a specific seam in the code; exclusion entry frames a priority decision	10
Live walkthrough	Can trace the full order path and the agent-offline re-plan path in code; conversations about design choices feel natural and considered	10
UI ceiling (optional)	Full dispatch board, SLA countdown, agent load, zone roster — scored proportionally to what's present and functional	+8
SSE streaming (bonus)	Token-by-token reasoning stream works end-to-end in the UI	+5

SUGGESTED TIMELINE

Six phases across five hours — treat these as rough checkpoints rather than a strict schedule. If you're meaningfully ahead or behind at any boundary, it's worth a moment to replan rather than just pushing forward.

Phase 1
~15-20 min

Read, sketch, decide

Read the full brief and the extensions roadmap. Sketch your architecture on paper before touching the IDE — entity relationships, routing pattern, how the agentic loop fires. Decide your frontend framework and your UI scope. Open your `ADR.md` now and write the first entry before the first line of code. The decisions you make here are the ones you'll explain in the walkthrough.

Phase 2
~60-75 min

Domain model, API, routing engine

Build the three entities with their state machines and JPA mappings. Wire the four endpoints and smoke-test them before moving on. Design the routing contract — interface first, then the rule-based implementation, then the runtime switchability mechanism. Write ADR entries for routing pattern and strategy switchability while they're fresh. Addendum A is ready for seed data.

Phase 3
~55-65 min

AI strategy and both prompts

Write both prompts in plain text before touching code — initial suggestion and re-plan are genuinely different documents. Then implement the AI strategy, using Addendum B for the HTTP layer. Build all fallback paths before testing the happy path. Test deliberately: break the LLM call and confirm the fallback works correctly. Write the AI resilience ADR entry.

Phase 4
~45-55 min

Agentic re-planning loop

Wire the event listener or application event. Implement the async re-plan — identify affected orders, run routing, persist suggestions with `triggerReason = AGENT_OFFLINE`. Test by PATCHing an agent to OFFLINE and watching suggestions appear. Test idempotency by triggering the same agent twice. Confirm the PATCH endpoint returns immediately without waiting for re-planning to finish. Write ADR-4.

Phase 5

Ops interface

~40-50 min

Scaffold your React or Angular project and build the floor: reassignment list with inline AI reasoning, accept/reject controls, re-plan badge, agent list, polling or manual refresh, loading states. Test the full end-to-end path — create order, assign agent, PATCH agent to OFFLINE, watch re-plan suggestion appear, accept it. If that path works cleanly and time remains, extend toward the UI ceiling.

Phase 6

~30-40 min

ADR, demo, submission

Complete any remaining ADR entries — the extensibility and deliberate exclusions entries especially. Record your 5-minute demo showing the re-plan path end-to-end. Write a README that gets the app running in under 5 minutes. Push to GitHub, make the repo public, and submit the link before the session ends.

> IF YOU FIND YOURSELF RUNNING BEHIND
>
> Drop the SSE bonus first — it's additive and costs real time. Then drop the UI ceiling and focus on a clean, working floor. If the frontend is still taking too long, simplify it aggressively — a minimal but functional interface that shows the re-plan badge is more valuable than an elaborate one that cost you the agentic loop or the ADR. Those two are worth protecting to the end.

BACKEND

Java 17+
Spring Boot 3.x
Maven or Gradle
Spring Data JPA
Spring Validation
Spring Actuator
H2 (dev) or Postgres
CORS: `localhost:5173` (React)
CORS: `localhost:4200` (Angular)

LLM — CHOOSE ONE

Gemini 1.5 Flash
aistudio.google.com · free

Groq + Llama 3.1 8B
console.groq.com · free · fast

Ollama local
ollama.com · no key · 8 GB+ RAM

Expected LLM output:

```
{"agentId":"...","confidence":0.85,"reasoning":"..."}
```

FRONTEND — CHOOSE ONE

React 18

Vite · `npm create vite@latest`

Dev server: `localhost:5173`

Angular 17

Angular CLI · standalone API

Dev server: `localhost:4200`

Document your choice in the ADR.

UI ceiling scoring available with either.

SUBMISSION

Public GitHub repo

`/backend` and `/frontend` folders

`README.md` — setup under 5 min

`ADR.md` — not optional

5-min demo video (Loom / YouTube)

Demo must show the re-plan path

Submit via recruiter form

REFERENCE · ALWAYS AVAILABLE

Two addenda provided to save you setup time on things that don't carry evaluation signal. They're here at the bottom of the brief so you can refer back to them at any point during the session without navigating away from your current tab. Use them as-is, adapt them to your schema and package structure, or ignore them — they don't constrain any of your design decisions.

ADDENDUM A — SEED SCRIPT

SQL · H2 AND POSTGRES COMPATIBLE

```
-- ─────────────────────────────────────────────────────
-- Hackathon · Seed Data
-- H2: place in src/main/resources/data.sql
-- Postgres: run manually or via Flyway
-- Adapt table/column names to match your schema
```

```
-- ──────────────────────────────────────────────────────────

-- 5 agents: mix of AVAILABLE and BUSY
INSERT INTO agents (id, name, active_order_count, status) VALUES
  ('AGT-001', 'Priya Sharma',  2, 'BUSY'),
  ('AGT-002', 'Rahul Verma',   0, 'AVAILABLE'),
  ('AGT-003', 'Ananya Iyer',   1, 'BUSY'),
  ('AGT-004', 'Kiran Nair',    0, 'AVAILABLE'),
  ('AGT-005', 'Deepak Mehta',  3, 'BUSY');

-- 8 orders: all ASSIGNED, spread across agents
INSERT INTO orders (id, description, assigned_agent_id, status, created_at) VALUES
  ('ORD-001', 'Electronics — Koramangala to Indiranagar', 'AGT-001', 'ASSIGNED', NOW()),
  ('ORD-002', 'Groceries — HSR Layout to BTM',          'AGT-001', 'ASSIGNED', NOW()),
  ('ORD-003', 'Pharma — Whitefield to Marathahalli',      'AGT-003', 'ASSIGNED', NOW()),
  ('ORD-004', 'Documents — MG Road to Jayanagar',        'AGT-005', 'ASSIGNED', NOW()),
  ('ORD-005', 'Food — Bellandur to Electronic City',      'AGT-005', 'ASSIGNED', NOW()),
  ('ORD-006', 'Apparel — Malleshwaram to Rajajinagar',    'AGT-005', 'ASSIGNED', NOW()),
  ('ORD-007', 'Books — Banashankari to JP Nagar',         'AGT-003', 'ASSIGNED', NOW()),
  ('ORD-008', 'Hardware — Peenya to Yeshwanthpur',        'AGT-001', 'ASSIGNED', NOW());
```

> **ADAPTING THIS TO YOUR SCHEMA**
>
> Align table names, column names, and identifier format with your own schema. For H2, drop this in `src/main/resources/data.sql` and Spring Boot runs it automatically on startup. For Postgres, run it manually or wire it through Flyway.

ADDENDUM B — LLM GATEWAY UTILITY

JAVA · GEMINI · GROQ · OLLAMA

```
package com..ai;
```

```java
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;
import java.util.List;
import java.util.Map;

/**
 * Handles the provider-specific HTTP wire format for Gemini Flash,
 * Groq, and Ollama. Returns the raw text content of the model's
 * response as a String.
 *
 * What this handles: authentication headers, request body shape,
 * and response unwrapping for all three supported providers.
 *
 * What remains yours: prompt construction, JSON parsing of the
 * returned String, agent ID validation, fallback logic, and the
 * distinction between your initial and re-plan calls.
 *
 * Configure in application.properties:
 *   llm.provider  = gemini          # gemini | groq | ollama
 *   llm.api-key   = ${LLM_API_KEY}  # not needed for ollama
 *   llm.model     = gemini-1.5-flash
 *   llm.base-url  = https://generativelanguage.googleapis.com
 */
@Component
public class LLMGateway {

    @Value("${llm.provider}")
    private String provider;

    @Value("${llm.api-key:}")
    private String apiKey;

    @Value("${llm.model}")
```

```
private String model;

@Value("${llm.base-url}")
private String baseUrl;

private final RestClient http = RestClient.create();

/**
 * Sends prompt to the configured LLM and returns the raw text
 * response. Throws RuntimeException on HTTP error or unparseable
 * response — caller is responsible for fallback handling.
 */
public String callLLM(String prompt) {
    return switch (provider.toLowerCase()) {
        case "gemini" -> callGemini(prompt);
        case "groq"   -> callOpenAICompatible(prompt,
                             baseUrl + "/openai/v1/chat/completions");
        case "ollama" -> callOpenAICompatible(prompt,
                             baseUrl + "/v1/chat/completions");
        default      -> throw new IllegalStateException(
                             "Unknown provider: " + provider);
    };
}

private String callGemini(String prompt) {
    var url  = baseUrl
        + "/v1beta/models/" + model
        + ":generateContent?key=" + apiKey;
    var body = Map.of("contents", List.of(
        Map.of("parts", List.of(
            Map.of("text", prompt)))));

    var resp = http.post().uri(url)
        .contentType(org.springframework.http.MediaType.APPLICATION_JSON)
```

```
                .body(body).retrieve().body(Map.class);

            try {
                var candidates = (List<?>) resp.get("candidates");
                var content    = (Map<?,?>)((Map<?,?>) candidates.get(0)).get("content");
                var parts      = (List<?>) content.get("parts");
                return (String)((Map<?,?>) parts.get(0)).get("text");
            } catch (Exception e) {
                throw new RuntimeException("Gemini response parse failed", e);
            }
        }

        private String callOpenAICompatible(String prompt, String url) {
            var body = Map.of(
                "model",    model,
                "messages", List.of(Map.of(
                    "role",    "user",
                    "content", prompt)));

            var resp = http.post().uri(url)
                .contentType(org.springframework.http.MediaType.APPLICATION_JSON)
                .header("Authorization", "Bearer " + apiKey)
                .body(body).retrieve().body(Map.class);

            try {
                var choices = (List<?>) resp.get("choices");
                var message = (Map<?,?>)((Map<?,?>) choices.get(0)).get("message");
                return (String) message.get("content");
            } catch (Exception e) {
                throw new RuntimeException("LLM response parse failed", e);
            }
        }
    }
```

GETTING STARTED WITH THIS

Drop it into whichever package fits your structure — the package declaration is a placeholder. Set the four `application.properties` values, call `callLLM(yourPrompt)`, and you get back a raw String. Parsing it as JSON, validating the agent ID, and deciding what to do when it throws are all yours to design.

KEEP YOUR API KEY OUT OF THE REPO

Store it as an environment variable and read it via `@Value("${llm.api-key}")`. A public GitHub repo with a committed API key will have it revoked within minutes of pushing.